

Exposing the Performance Hidden Costs: An Empirical Study of Application-Layer Overhead in High-Throughput gRPC Microservices

Irawan Widi Widayat**
Toyohashi Institute of Technology
Toyohashi, Japan
irawan@perf.cs.tut.ac.jp

Yukinori Sato
Toyohashi Institute of Technology
Toyohashi, Japan
yukinori@cs.tut.ac.jp

Valerie Béranger
Inria Paris-Rocquencourt
Rocquencourt, France

Abstract

Current development of microservice architectures heavily relies on API gateways and high-performance RPC frameworks like gRPC, the implementation details of the inter-service communication layer are often abstracted away in system design. Our paper empirically investigates the performance degradation caused by application-layer communication bottlenecks, with particular focus on gRPC connection management. We present a mathematical queueing model and algorithmic formalization to quantify the overhead of the connection-per-request compared to a persistent multiplexed channel. Our evaluation results in Kubernetes environment tests show that fixing the single implementation flaw can significantly reduce median latency by 97%—from over 7,000 ms to 150 ms under peak load—moving the performance bottleneck from the infrastructure gateway back to the compute backend.

CCS Concepts

• **Networks** → **Network performance analysis**; *Cloud computing*; • **Software and its engineering** → *Interoperability*.

Keywords

Microservices, Cloud-Native, Kubernetes, High Performance Computing, gRPC API

ACM Reference Format:

Irawan Widi Widayat, Yukinori Sato, and Valerie Béranger. 2018. Exposing the Performance Hidden Costs: An Empirical Study of Application-Layer Overhead in High-Throughput gRPC Microservices. In *Proceedings of Make sure to enter the correct conference title from your rights confirmation email (Conference acronym 'XX)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/XXXXXXX.XXXXXXX>

1 Introduction

The Broad Context and Problem. The transition to cloud-native microservices has established gRPC over HTTP/2 as a standard for

inter-service communication; however, API gateways often introduce severe, undocumented latency overheads and dependencies [5]. A study by [8] has extensively documented that interservice communication overhead is a primary performance bottleneck in distributed systems, often overshadowing the execution time of the business logic itself. While theoretical architectural models assume that gateway-to-backend communication operates at near-optimal efficiency, these models often overlook how the application-layer implementation interacts and governs client-server communication. Even though HTTP/2 is designed to mitigate latency via connection multiplexing [6], real-world gateway implementations frequently fail to utilize this capability properly. This can lead to a structural protocol efficiency loss, and creating a common performance penalty at the edge of the microservice cluster.

The specific issue and prior work. This paper discusses a problem related to performance slowdowns caused by the "connection-per-request" (CPR) method used in API gateways. Previous studies [3] have shown how TCP and TLS handshake delays reduce throughput in distributed systems. The community recognizes that connection pooling is essential for older HTTP/1.1 and database systems. However, using these traditional connection management techniques in HTTP/2 setups is problematic. Opening a new connection for each incoming request is an $O(N)$ operation, which clashes with HTTP/2's efficient $O(1)$ design. This inefficiency makes gateway worker threads waste CPU power on switching contexts instead of handling requests. This poor design can lead to unpredictable delays in containerized environments, harming system scalability.

The gap and our contribution. Even though researchers have extensively modeled microservice performance, the specific intersection of application-layer connection churn and queueing network saturation remains empirically unquantified. We define connection churn as the rapid, continuous cycle of establishing and tearing down complex network state—specifically TCP handshakes and HTTP/2 protocol negotiations—for every individual application-layer request, rather than multiplexing traffic over a persistent channel. A recent study that utilizes $M/M/1$ and $M/M/c$ stochastic queueing models to evaluate microservice resilience was introduced by Pinheiro et al. [7]. However, these models typically assume that network speed remains constant. We argue that implementation-level flaws, especially for gRPC connection churn, dynamically inflate the service rate. This churn converts what should be a constant-time RPC execution into a highly variable, load-dependent event [2]. Previous studies haven't shown how these connection problems lead to a dramatic increase in latency before the system's limits are

*Both authors contributed equally to this research.

Unpublished working draft. Not for distribution.

Permission to make digital or hard copies of all or part of this work for personal or internal use, or the internal or personal use of specific clients, is granted by ACM for libraries and registered users, provided that the fee of \$12.00 is paid directly to ACM. This permission is granted without fee to individuals and their employers for non-commercial use, provided that they acknowledge ACM and its copyright, and that they pay the fee of \$12.00 per copy to ACM. For all other use, permission should be sought from ACM. Copyright © 2018 ACM 978-1-4503-XXXX-X/2018/06 <https://doi.org/XXXXXXX.XXXXXXX>

© 2018 Copyright held by the owner/author(s). Publication rights licensed to ACM. ACM ISBN 978-1-4503-XXXX-X/2018/06 <https://doi.org/XXXXXXX.XXXXXXX>

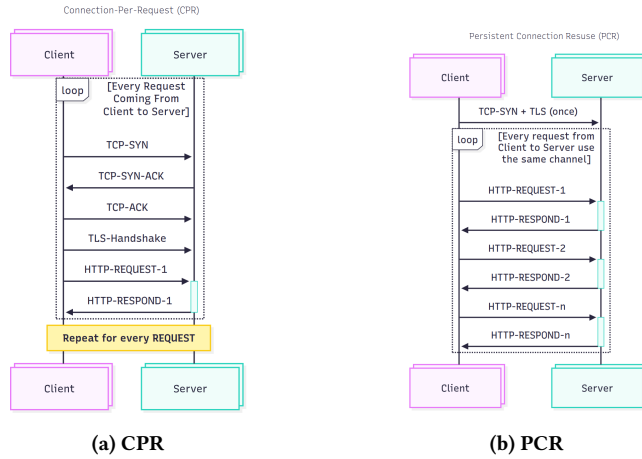


Figure 1: Two framework models of gRPC connections mechanism. Connection-Per-Request (CPR) vs Persistent Connection Reuse (PCR).

reached. Henceforth, bridging the gap between theoretical queuing models and empirical application-layer protocol behavior is necessary to accurately predict cloud-native performance.

Roadmap and finding. In this paper, we formalize the latency and throughput costs of gRPC connection churn and propose an optimized persistent-channel alternative. We introduce a queueing-theoretic model to explain the non-linear latency spikes observed under high load and validate this model through strict component-isolation benchmarking within a Kubernetes environment. Our empirical results demonstrate that correcting this specific anti-pattern yields a 97% reduction in median system latency under saturation. Ultimately, this paper demonstrates to the research community that application-level protocol correctness is a mandatory prerequisite for the validity of macroscopic cloud performance models, proving that implementation details are as critical to scalability as the underlying architectural design.

2 Mathematical Modeling of gRPC Connection Management

In order to comprehend the communication bottleneck, it is essential to break down the entire lifecycle of a request as it progresses through the API gateway. This analysis will help us identify the key stages and factors contributing to the bottleneck in the communication process.

Let T_{total} be the total end-to-end latency experienced by the client. We define T_{total} as:

$$T_{total} = T_{http} + W_{queue} + S_{gateway} + T_{network} \quad (1)$$

Where:

T_{http} : ingress HTTP processing time.

W_{queue} : queueing delay at the gateway's worker processes.

$S_{gateway}$: service time required by the gateway to establish and execute the downstream gRPC call.

$T_{network}$: physical network propagation delay and backend computation time (baseline constant $c \approx 3$ ms).

The critical variable in our study is $S_{gateway}$. We model this under two distinct operational paradigms.

2.1 Paradigm 1: Connection-Per-Request (CPR)

In the CPR anti-pattern, as shown in the Fig. 1a, the gateway initializes a new gRPC channel for every incoming request. The service time S_{cpr} is defined as:

$$S_{cpr} = t_{tcp} + t_{http2} + t_{rpc} + t_{teardown} \quad (2)$$

Where t_{tcp} is the TCP 3-way handshake, t_{http2} is the HTTP/2 SETTINGS frame negotiation, t_{rpc} is the actual payload serialization/transmission, and $t_{teardown}$ is the TCP teardown (FIN/ACK). Because t_{tcp} and t_{http2} require multiple network round-trip times (RTTs) and significant CPU context switching, $S_{cpr} \gg t_{rpc}$.

2.2 Paradigm 2: Persistent Channel Reuse (PCR)

In the optimized PCR pattern, as shown in Fig. 1b, the gateway initializes a single, thread-safe gRPC channel during application startup. All incoming requests are multiplexed over this channel. The setup costs are amortized over millions of requests, approaching zero per request. Thus, the service time S_{pcr} simplifies to:

$$S_{pcr} = t_{rpc} \quad (3)$$

2.3 Queueing Theory and Non-Linear Saturation

The catastrophic latency spikes observed in empirical testing (e.g., jumping to 7,000 ms) cannot be explained by S_{cpr} alone. We must apply an $M/M/c$ queueing model, where incoming requests arrive at rate λ and are serviced by c gateway worker threads at rate $\mu = 1/S_{gateway}$.

The utilization of the gateway system is $\rho = \frac{\lambda}{c \cdot \mu}$. According to Little's Law [4] and standard queueing theory, the average queueing delay W_{queue} grows asymptotically as utilization $\rho \rightarrow 1$:

$$W_{queue} \propto \frac{\rho}{1 - \rho} \quad (4)$$

Because S_{cpr} is significantly larger than S_{pcr} , the service rate μ_{cpr} is artificially low. Therefore, under the CPR paradigm, the system reaches critical utilization ($\rho \rightarrow 1$) at a much lower arrival rate λ (e.g., 400 RPS). When λ exceeds this threshold (e.g., 600 RPS), W_{queue} approaches infinity, resulting in the massive latency spikes and connection timeouts observed in our baseline data.

3 Algorithmic Formalization

The primary objective of Algorithm 1 is to map the abstract queueing variables from Section 2 directly to their application-layer implementations. It illustrates exactly where gateway service time ($S_{gateway}$) is consumed by contrasting both configurations.

Under the Connection-Per-Request (CPR) paradigm, channel initialization and teardown occur locally within the request handler (Lines 10-12, 17-19). This imposes an $O(N)$ computational penalty, manifesting as the "connection churn" that degrades the service rate (μ). Conversely, the Persistent Channel Reuse (PCR) paradigm shifts this network state management to a global, one-time execution at startup (Lines 2-5). By referencing a thread-safe, multiplexed

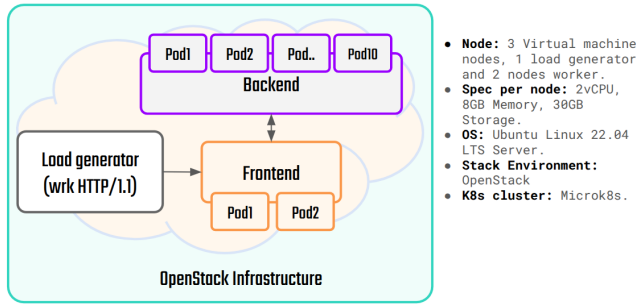


Figure 2: Kubernetes cluster environment.

channel ($O(1)$ overhead), the structural refactoring averts queue saturation without altering the core RPC business logic.

Algorithm 1 API Gateway Request Handler: CPR vs. PCR Paradigm

Require: req_number : Integer, $mode$: {CPR, PCR}

Ensure: is_prime_result : Boolean

```

1: // PCR Global Initialization (Executed Once at Startup,  $O(1)$ )
2: if  $mode == \text{PCR}$  then
3:    $global\_channel \leftarrow \text{grpc.insecure\_channel}(\text{SERVER\_ADDRESS})$ 
4:    $global\_stub \leftarrow \text{PrimeCheckerStub}(global\_channel)$ 
5: end if
6: function  $\text{HandleRequest}(req\_number, mode)$ 
7: if  $mode == \text{CPR}$  then
8:   // CPR Local Initialization (Executed per request,  $O(N)$ )
9:    $channel \leftarrow \text{grpc.insecure\_channel}(\text{SERVER\_ADDRESS})$ 
10:   $stub \leftarrow \text{PrimeCheckerStub}(channel)$ 
11: else
12:   $stub \leftarrow global\_stub$  //  $O(1)$  Memory Reference
13: end if
14:  $payload \leftarrow \text{IsPrimeRequest}(number = req\_number)$ 
15:  $response \leftarrow stub.\text{Check}(payload)$  // Network I/O
16: if  $mode == \text{CPR}$  then
17:    $channel.close()$  // Teardown overhead
18: end if
19: return  $response.is\_prime$ 
20: end function

```

4 Methodology

To validate the proposed mathematical model and pinpoint accurately where latency overhead originates, we conducted controlled experiments in a containerized distributed setting. Our evaluation follows a component isolation strategy: we first measure end-to-end performance across the full system and then benchmark the backend service alone in order to distinguish computation time from communication-related delays.

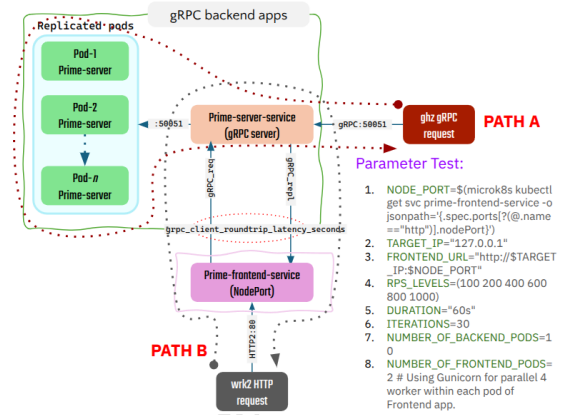


Figure 3: Evaluation mechanism to accurately examine the frontend (HTTP) and backend (gRPC) latency utilizing different benchmark tools.

4.1 Experimental Environment and Architecture

The experiment was conducted on a Kubernetes cluster composed of multiple worker nodes as shown in the Fig. 2. The system under test (SUT) consists of two main microservice components:

- **gRPC Backend Service (prime-server):** A high-performance Python application responsible for CPU-intensive prime number verification. To eliminate backend capacity as a confounding variable, this component was over-provisioned with 10 identical replica pods, deployed behind a Kubernetes NodePort Service.
- **API Gateway (prime-frontend):** A Python Flask application running within Unicorn workers. This component acts as an HTTP/REST to gRPC proxy. It was strictly limited to 2 replica pods to intentionally force saturation and observe gateway-tier queuing dynamics under heavy load.

4.2 Workload Generation and Component Isolation

To precisely calculate the application-layer gateway overhead, we utilized two specialized, open-source load-testing tools, as shown in the Fig. 3:

- **Unoptimized Gateway (CPR):** Simulates Algorithm 1 ($mode = \text{CPR}$), forcing TCP/HTTP2 renegotiation for every incoming request.
- **Optimized Gateway (PCR):** Simulates Algorithm 1 ($mode = \text{PCR}$), routing concurrent accesses through a single, globally initialized gRPC channel.

Both tools executed 30 iterations for each target throughput level (ranging from 100 to 1000 Requests Per Second, or RPS), sustaining the load for 60 seconds per iteration to allow queueing buffers to stabilize.

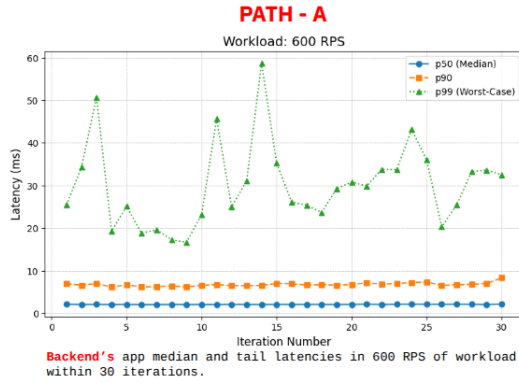


Figure 4: gRPC backend latency evaluation using ghz benchmark tool.

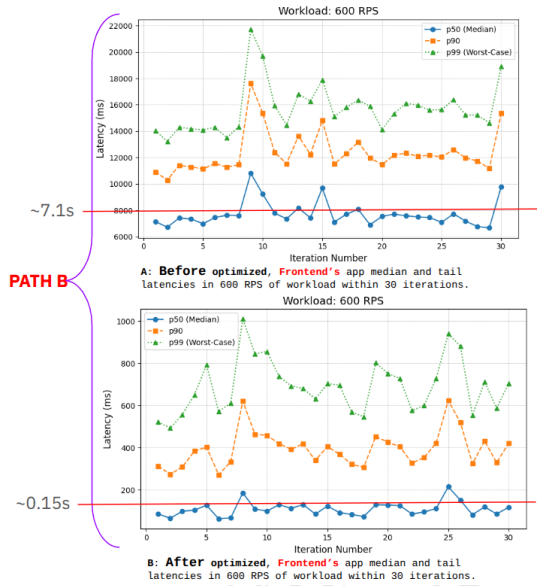


Figure 5: End-to-end frontend latency evaluation using wrk2 as workload generator. (A) Connection-Per-Request paradigm - Before optimized, (B) Persistent Channel Reuse paradigm - After optimized

5 Results and Discussion

The empirical results have successfully proven that the predicted Asymptotic latency degradation is caused by the $M/M/c$ queuing model in the Connection-Per-Request (CPR) paradigm and have clearly demonstrated the transformational impact of Persistent Channel Reuse (PCR).

5.1 Baseline: Backend gRPC Performance

The ghz test results established an exceptional and highly stable baseline. Across the entire load spectrum (100 to 1000 RPS), the 10 prime-server backend pods consistently delivered a median (p50)

latency of ~ 2.5 ms to 3.6 ms, with p90 latencies remaining firmly under 10 ms.

This flat latency curve definitively proved that the backend architecture was not the system bottleneck, as shown in the Fig. 4. By having 10 backend pods, we mathematically guaranteed that the backend possessed vastly more computational capacity than the frontend could ever saturate. The computational capacity of the prime number verification logic was highly resilient, isolating the source of any overarching performance degradation to the gateway tier.

Concurrency Configuration and Multiplexing: During empirical testing, both *ghz* and *wrk2* were configured to simulate high concurrency using a fixed pool of 50 simultaneous client connections across all target throughput levels (λ). It is critical to note how these concurrent accesses are handled internally by the optimized gateway. Under the PCR paradigm, the concurrent requests arriving at the gateway are not queued sequentially over the network; instead, the underlying gRPC framework maps these concurrent application-level accesses into discrete, parallel HTTP/2 streams over a *single* shared TCP connection. This mechanism prevents connection exhaustion and directly validates our mathematical assumption that S_{per} approaches pure RPC execution time without queuing at the network socket layer.

5.2 Unoptimized System: The CPR Bottleneck

Testing the end-to-end system utilizing the CPR paradigm revealed a catastrophic bottleneck, as shown in the Fig. 5-A (upper side). At a moderate load of 400 RPS, the system maintained a median latency of ~ 400 ms. However, as throughput increased to 600 RPS, the system hit the critical utilization threshold ($\rho \rightarrow 1$).

Median latency exploded to $> 7,100$ ms, and p99 latency spiked past 14,000 ms. This exponential degradation perfectly aligns with the queuing theory model where service time (S_{cpr}) is artificially inflated by excessive TCP/HTTP2 handshake overheads for every transaction. The gateway workers were entirely occupied managing network state rather than processing business logic.

5.3 Optimized System: PCR Implementation

By deploying the algorithmic optimization detailed in Section 3 (establishing a persistent, multiplexed gRPC channel at application startup - Algorithm ??), the performance profile of the system was qualitatively transformed.

At 600 RPS, the optimized system recorded a median latency of ~ 150 ms, as shown in the Fig. 5-B (lower side). This represents a $\sim 97.9\%$ reduction in end-to-end latency compared to the CPR paradigm. Furthermore, the system successfully sustained loads up to 800 RPS before demonstrating signs of stress, effectively doubling the system's maximum throughput capacity without provisioning any additional infrastructure resources.

5.4 Quantifying the Inherent Proxy Latency Gap

By subtracting the baseline backend network latency ($T_{network}$) established via *ghz* from the full end-to-end system latency (T_{total}) measured via *wrk2*, we can isolate the underlying application-layer

tax. We define this remaining overhead as the **Inherent Proxy Latency Gap**—the baseline latency strictly attributable to the framework’s internal request handling, independent of network state management.

Table 1: Comparative Latency Overhead at Optimal Saturation Point (400 RPS)

Component Level	Median Latency (p50)	Percentage of Total Latency
Backend Only (ghz)	~ 2.8 ms	2.5%
Full System (CPR, Before)	~ 400 ms	100%
Full System (PCR, After)	~ 110 ms	100%
Gateway Overhead (PCR)	~ 107.2 ms	97.5%

As illustrated in Table 1, the results reveal a compelling paradox in the optimized PCR paradigm. Although the persistent channel implementation successfully averted queue saturation and achieved a 97% reduction in total latency compared to the CPR baseline, a compositional analysis shows that the gateway tier still accounts for **97.5% of the remaining end-to-end response time**.

This empirical gap directly connects our theoretical approach to the user experience. In our PCR model, the actual backend computational execution ($T_{network}$) requires only ~ 2.8 ms (2.5% of total time), while the remaining 107.2 ms is consumed by the gateway. Critically, this gap encompasses not only the Python WSGI [1] stack execution but also the **protocol transcoding overhead**—the computational cost of parsing incoming HTTP/1.1 JSON requests from *wrk2* and serializing them into HTTP/2 Protobuf messages for the gRPC backend.

This reveals a fundamental "Performance-Abstraction Paradox." While developers utilize API gateways to seamlessly bridge RESTful clients (HTTP/1.1) with high-performance microservices (gRPC), this protocol translation introduces a structural latency tax that dwarfs the business logic itself. Consequently, while our PCR optimization resolves protocol-level queue saturation, the absolute lower bound of system latency remains strictly dictated by the gateway’s HTTP-to-gRPC transcoding efficiency.

5.5 Empirical Validation of the Queueing Model

To rigorously evaluate the mathematical framework proposed in Section 2, we mapped our empirical throughput and latency data to the $M/M/c$ queueing model. Our objective was to prove that the catastrophic latency spikes observed were not random system failures, but rather predictable queueing saturation events driven by connection churn.

In our system architecture, the gateway tier acts as the server pool (c workers). The arrival rate of incoming requests is defined as λ (varying from 100 to 1000 RPS). The service rate of the gateway, μ , is inversely proportional to the service time ($S_{gateway}$).

Under the Connection-Per-Request (CPR) paradigm, the service time (S_{cpr}) includes the mandatory TCP and HTTP/2 handshakes ($t_{tcp} + t_{http2}$). Because these network operations block gateway worker threads, μ_{cpr} is severely restricted.

Our empirical data aligns perfectly with the theoretical utilization formula:

$$\rho = \frac{\lambda}{c \cdot \mu}$$

According to Little’s Law [4], queueing delay W_{queue} grows asymptotically as system utilization $\rho \rightarrow 1$:

$$W_{queue} \propto \frac{\rho}{1 - \rho}$$

Evaluating the CPR Saturation Point: During our baseline CPR testing, the system maintained a median latency of ~ 400 ms at $\lambda = 400$ RPS, indicating high but stable utilization ($\rho < 1$). However, when the arrival rate increased to $\lambda = 600$ RPS, the median latency violently spiked to > 7,100 ms. This exponential degradation confirms that the low service rate (μ_{cpr}) caused the gateway to cross the critical threshold of $\rho = 1$ at a mere 600 RPS. At this point, requests were arriving faster than connections could be negotiated, causing W_{queue} to theoretically approach infinity, which manifested empirically as 7-second tail latencies and connection timeouts.

Evaluating the PCR Optimization: By transitioning to the Persistent Channel Reuse (PCR) paradigm, we eliminated the handshake overhead, drastically reducing the service time to S_{pcr} and thereby significantly increasing the service rate (μ_{pcr}). Because $\mu_{pcr} \gg \mu_{cpr}$, the utilization ρ at the same arrival rate ($\lambda = 600$ RPS) remained well below 1.

Consequently, the queueing delay W_{queue} collapsed. The empirical result—a drop in latency from > 7,100 ms to exactly 150 ms at 600 RPS—serves as a direct physical validation of the $M/M/c$ model. It proves that correcting the $O(N)$ connection anti-pattern artificially scales the service rate μ , averting queue saturation without provisioning any additional hardware capacity (c).

6 Conclusion and Future Work

This study demonstrates how small details in application-layer implementation can seriously disrupt cloud-native systems that need to scale. We found that a common mistake—creating new connections for each request—can cut system throughput by 50% and increase response time by more than 7,000 ms. We improved the queueing issue by optimizing to a Persistent Channel Reuse (PCR) models. However, our method for isolating components showed that the application-layer proxy adds about ~ 100 ms of overhead. This data leads to the main conclusion: ensuring the correct implementation is just as important as having a scalable architecture. Additionally, infrastructure tasks like API routing should not be managed by general application frameworks.

Future work will expand this steady-state benchmarking methodology into the domain of Chaos Engineering. By introducing controlled faults (e.g., node resource starvation, sudden pod termination) under a persistent load.

Acknowledgments

This work is based on results obtained from the project, "Research and Development Project of the Enhanced infrastructures for Post-5G Information and Communication Systems" (JPNP20017), commissioned by the New Energy and Industrial Technology Development Organization (NEDO).

References

- [1] Phillip J. Eby. 2010. PEP 333 – Python Web Server Gateway Interface v1.0. Internet Article. Retrieved April 23, 2026 from <https://peps.python.org/pep-0333/>
- [2] Robert Heinrich, André Van Hoorn, Holger Knoche, Fei Li, Lucy Ellen Lwakatare, Claus Pahl, Stefan Schulte, and Johannes Wettinger. 2017. Performance engineering

- for microservices: research challenges and directions. In *Proceedings of the 8th ACM/SPEC on international conference on performance engineering companion*. 223–226.
- [3] Kasun Indrasiri and Danesh Kuruppu. 2020. *gRPC: up and running: building cloud native applications with Go and Java for Docker and Kubernetes*. O'Reilly Media.
- [4] John DC Little. 2011. Or forum—Little's law as viewed on its 50th anniversary. *Operations research* 59, 3 (2011), 536–549.
- [5] Alberto Martin-Lopez, Sergio Segura, and Antonio Ruiz-Cortés. 2022. Online testing of RESTful APIs: Promises and challenges. In *Proceedings of the 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. 408–420.
- [6] Liang Ming, Y. Leau, and Ying Xie. 2024. Distributed Denial of Service Attack in HTTP/2: Review on Security Issues and Future Challenges. *IEEE Access* 12 (2024), 33296–33308. <https://api.semanticscholar.org/CorpusId:268151580>
- [7] Thiago Pinheiro, Marco Mialaret, Paulo Pereira, Luan Lins, Daliton Silva, Jamilson Dantas, and Paulo Maciel. 2024. Performance modeling of microservices with circuit breakers using stochastic petri nets. In *2024 IEEE International Systems Conference (SysCon)*. IEEE, 1–8.
- [8] Xiangfeng Zhu, Guozhen She, Bowen Xue, Yu Zhang, Yongsu Zhang, Xuan Kelvin Zou, XiongChun Duan, Peng He, Arvind Krishnamurthy, Matthew Lentz, et al. 2023. Dissecting overheads of service mesh sidecars. In *Proceedings of the 2023 ACM Symposium on Cloud Computing*. 142–157.